

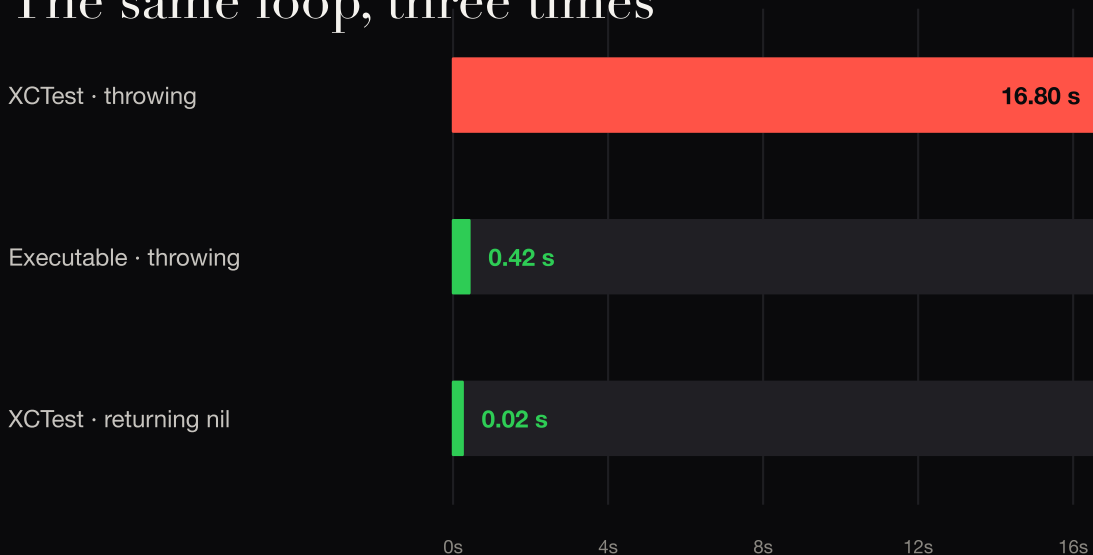
The hidden cost of throw

Why your fuzzer crawls inside XCTest

IN SHORT

Throwing errors in a tight loop under XCTest is roughly 40 times slower than the same code in an executable. XCTest observes every throw and bridges it to an NSError. try? does not help. If you fuzz, run property tests, or benchmark code that throws often, this is quietly costing you hours.

The same loop, three times



20 million calls, half of them throw. Same work; only the context changes.

40×

slower when throwing

~1.6 μs

cost per throw

850×

returning nil cures it

Where this came from

This did not surface in a toy micro-benchmark. It came out of a real workload, and that scale is precisely what exposes it.

4.8 bn	8	10	3	2 h 37
inputs	doors	workers	throw	the run

Kalego is an encrypted messenger. Its client has eight 'doors' that receive data from the network: parsers and decoders that, by design, take rubbish chosen by an adversary as the NORMAL case. Each was hit with 600 million malformed inputs, in parallel across ten cores.

The crucial detail: three of those eight doors THROW on rubbish, and rightly so, be it a hostile proxy response, a corrupt ratchet message or a tampered vault. So 'throwing on rubbish' was not rare: it was the common case, hundreds of millions of times over.

WHAT FORCED A CLOSER LOOK

The first attempt stalled: one core at 100 per cent for over an hour whilst the other nine sat idle. No crash, no warning. At the scale of an ordinary test, a handful of throws, nobody would ever see it; at fuzzing scale, it is catastrophic.

The symptom: it looks like a hang

This throws no error and no crash. It shows up as slowness, and misleading slowness at that:

- **A single thread at 100 per cent.**

In a parallel test, nine cores finish and one is left alone, flat out, for minutes or hours. It looks like an ordinary straggler, but it isn't keeping pace.

- **The wall-clock time balloons.**

A run that ought to take minutes takes hours. No message, no clue.

- **No tool complains.**

No crash, no leak, no warning. Memory sits flat. Everything 'works', only agonisingly slowly.

- **It scales with the number of throws.**

The more exceptions the loop throws, the worse it gets. A small test never notices; one that throws millions of times does.

WHERE IT SHOWS UP

Fuzzing. Property-based testing (SwiftCheck and the like). Stress tests. Micro-benchmarks. Any loop inside XCTest that repeatedly calls code which throws, be it a parser, a decoder or a validator.

Diagnosis: see it for yourself

Since there is no error, you have to look at what the stuck thread is actually doing. Without stopping the process, sample its call stacks with the macOS 'sample' tool:

```
# find the PID of the test process
pgrep -f xctest

# sample for 5 seconds and save the stacks
sample <PID> 5 -f /tmp/sample.txt
```

The stack of the running thread gives the cause away. This is what comes out, from the top (deepest) downwards:

```
tu bucle / your loop
parse(...)
  swift_willThrowTypedImpl          <- willThrow
  XCTSwiftErrorObservation._installErrorObserver
    _swift_stdlib_bridgeErrorToNSError
      _getErrorUserInfoNSDictionary
        ...CFBasicHashFindBucket, isEqualToString...
```

In plain terms: every time your code throws, XCTest intercepts it and builds a full NSError, complete with its userInfo dictionary. That is the whole cost, and none of it is yours.

Why it happens

XCTest wants to warn you when a test throws an error you did not expect. To do that it installs a global observer in the Swift runtime, hooked onto 'willThrow': the point through which every exception passes at the moment it is thrown.

On each throw, that observer bridges the Swift error to an Objective-C NSError: it allocates memory, builds the userInfo dictionary, compares strings. For an ordinary test that throws once or twice it is imperceptible. For a loop that throws millions of times it is ruinous: we measured about 1.6 microseconds of pure overhead per throw.

THE DETAIL THAT CATCHES YOU OUT

try? and do/catch will not save you. The observer fires ON the throw, before your catch receives the error. It makes no difference that you catch it at once: the cost was already paid at the 'throw'. Simply ignoring the exception is not enough.

In a plain executable, or in the shipping app, that observer does not exist: nobody is watching the throws. That is why the same code flies outside XCTest.

```
// This does NOT avoid the cost: the throw already happened.
for x in inputs {
    _ = try? parse(x)      // <- still paying the observer
}
```

Reproduce it in two minutes

A function that throws for half of its inputs, like a parser fed rubbish, and the same loop in two places:

```
// Lib.swift
public enum ParseError: Error { case malformed }
@inline(never)
public func parse(_ x: Int) throws -> Int {
    if x & 1 == 0 { throw ParseError.malformed }
    return x
}

// the loop, identical in the test and the executable:
for i in 0..<20_000_000 { _ = try? parse(i) }
```

Measured on a MacBook Pro M5, Swift 6.3.3, built in release:

CONTEXT	TIME	RELATIVE
Executable (swift run)	0.42 s	×1
XCTest (swift test)	16.80 s	×40
XCTest, parseOpt returns nil	0.02 s	×0.05

Same loop, same work. The only variable is throw versus nil, and XCTest versus not.

How to fix it

✓ Move the hot loop out of XCTest.

Put it in an executable, or a benchmark target. With no observer, a throw costs next to nothing again. It is the best option for fuzzing and stress tests.

✓ Return an optional, do not throw.

On paths travelled millions of times, 'return nil' instead of 'throw' avoids the observer entirely. In the test: from 16.80 s to 0.02 s.

✓ If it must stay in XCTest, do not throw in the loop.

Restructure so the exception is genuinely exceptional, not the common case. Reserve throw for what should actually stop the test.

THE SHORT RULE

XCTest's error observer makes every throw cost about 1.6 microseconds. Fine for hundreds of throws; ruinous for millions. If you are going to throw a great deal, do it outside XCTest.

Measured on a MacBook Pro M5 · Swift 6.3.3 · macOS 26 · built in release. It surfaced whilst fuzzing the Kalego client: a run of 4.8 billion inputs stalled on one core for over an hour before this was found. Minimal reproduction and figures above. Do check the numbers on your own machine.